

K-BIG-2

The Structured, Viability-Scored Business Idea Engine
Kwacha Intelligence Platform (KIP) -- Technical & Investor Documentation

September 2026

1. What K-BIG-2 Is

K-BIG-2 is version 2 of KIP's business idea generation engine. Where K-BIG-1 answers a free-text chat prompt with a single markdown recommendation, K-BIG-2 collects a **structured profile** from the entrepreneur (available capital, location, skills, and assets already owned), generates three candidate ideas as validated structured data using the same Zambia-grounded reasoning as K-BIG-1, and -- critically -- scores every idea against a **7-variable weighted viability formula** before ranking it against KIP's entire accumulated idea dataset, not just the three ideas generated for this request.

The goal is to reduce two weaknesses in K-BIG-1: **bias** (a single LLM pass with no structured self-check on financial or market realism) and **viability rigor** (no numeric scoring of whether an idea is actually a good fit for this specific person's capital, skills, and location). K-BIG-2 addresses both by forcing the model to produce machine-checkable numbers and self-assessed sub-scores, then applying a deterministic formula on top -- and by growing a reusable, structured dataset every time anyone generates ideas, so the platform's recommendations compound in value over time.

2. The Structured Intake Flow

On the My Ideas page, a "New Idea" button opens a 4-slide wizard (shown one question at a time):

- **Slide 1 -- Available Capital.** Amount in Kwacha. Can be skipped; skipping asks whether a loan or grant is expected in future and in what range -- if none is expected, capital is treated as effectively limitless for scoring purposes.
- **Slide 2 -- Location.** Province/district/area, free text, or "Use my location" (device GPS reverse-matched to the nearest known town via the haversine formula against KIP's existing town coordinate registry -- no third-party geocoding dependency). Cannot be skipped.
- **Slide 3 -- Skills.** Multi-select across 9 categories (selling & marketing, operations & management, digital & creative, food & catering, agriculture, trades & repairs, services & care, finance & administration, transport & logistics).
- **Slide 4 -- Assets.** Multi-select, grouped into Physical, Digital, and Business assets already owned.

Submitting the final slide sends this profile to `POST /api/ideas/generate`, which runs the full pipeline described below.

3. Architecture & Data Flow

K-BIG-2 Data Flow

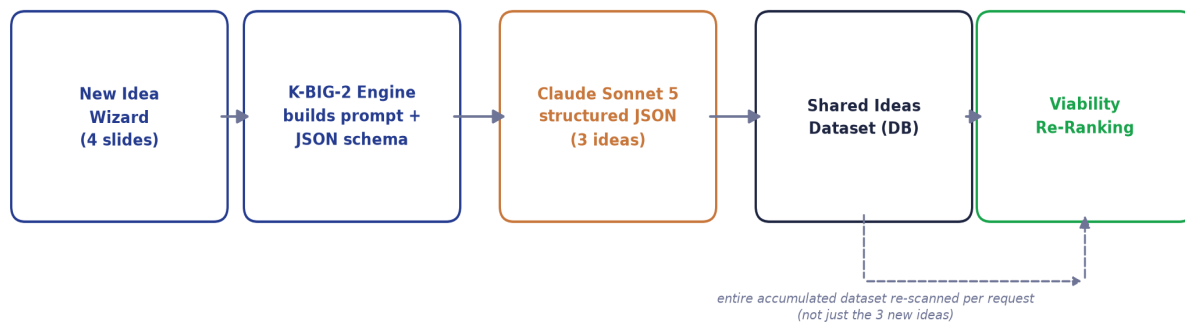


Figure 1 -- the k-big-2 request pipeline, from wizard submission to ranked results.

Per the sprint specification, every request to k-big-2 performs three steps: (1) generate 3 new business ideas using the K-BIG-1 method -- the same Zambia economic constants, town intelligence, and knowledge-base retrieval K-BIG-1 already assembles, now packaged for validated structured JSON output instead of a single markdown reply; (2) add those 3 ideas to a shared dataset, visible to every future request from any user, not just this one; and (3) open the *entire* dataset and re-rank it for this specific requester's viability, so a strong idea generated for someone else last month can resurface for a new requester with a similar profile. This is a deliberate data-network-effect design: the dataset gets smarter and more useful the more people use K-BIG-2.

4. The Structured Idea Schema

Every idea K-BIG-2 generates is captured as a structured object (not prose) spanning identity, user-fit, market, financial, operational, regulatory, location, strategic, and confidence fields -- following the sprint's practical schema. Each idea is persisted with:

Field group	Representative fields	Purpose
Identity	name, category, description, location_fit	What the business is
User-fit	min_capital, recommended_capital_range, required_skills, required_assets, risk_level	Can this person realistically run it
Market	demand_level, total_target_buyers, consumption_frequency, average_unit_price	Feeds the Demand (D) score
Financial	startup_cost, monthly_revenue_estimate, cost_of_goods_sold, break_even_months	Feeds the Financial (F) score
Operational / Regulatory / Location	premises, staffing, licensing, urban/rural fit, home-based suitability	Feasibility & compliance
Strategic / Confidence	scalability, digital_potential, data_confidence, evidence_quality	Growth potential & self-reported certainty
Narrative	narrative (free text)	Everything else the sprint schema calls for (equipment, logistics, supplier dependency, seasonality, brand/upsell potential) in prose rather than 30+ more strict fields -- see §7 methodology note
Viability sub-scores	execution_fit_score, competitive_position_score, regulatory_risk_score, asset_location_score	Self-assessed 1-10, feed the formula

Every idea is also stored in full as one JSON object (the structured_data column), so nothing in the schema is lost even where it isn't promoted to its own database column for filtering, sorting, or CSV export.

5. Category CAGR Constants

The Demand formula (below) needs a compound annual growth rate per business category. Since no ready-made Zambian sector dataset exists, K-BIG-2 maintains a manually-updated constants file (category_cagr . json) mapping ~12 business categories to an estimated growth rate, refreshed monthly as better data becomes available. This is explicitly a modelling assumption, not an audited figure -- see the methodology notes in §7.

6. The Viability Formula

Every idea is scored 0-10 across seven weighted variables:

$$V = 0.25D + 0.20F + 0.15C + 0.15E + 0.10R + 0.10S + 0.05A$$

Var.	Name	Formula / basis	Computed by
D	Demand	$N \times Q \times P \times (1+g)^t$, log-scaled to 0-10	Arithmetic (viability_engine.py)
F	Financial viability	Gross margin % x 10	Arithmetic
C	Capital fit	(Capital Available / Capital Required) x 10	Arithmetic
E	Execution fit	User's skills/time vs. the idea's needs	LLM self-score (new ideas) / heuristic overlap (reused ideas)
S	Competitive position	Local competition & moat strength	LLM self-score (idea-intrinsic, reused unchanged for every requester)
R	Regulatory/operational risk	Licensing, compliance, supply fragility	LLM self-score (idea-intrinsic)
A	Asset & location advantage	Owned assets/location vs. the idea's needs	LLM self-score (new) / heuristic overlap (reused ideas)

Why C/E/A are recomputed per requester but D/F/S/R are not: demand, financial viability, competitive position, and regulatory risk are properties of the *business idea itself* -- they don't change depending on who is asking. Capital fit, execution fit, and asset/location advantage are properties of the *match between the idea and a specific person's resources*. This split is what makes dataset-wide re-ranking (§3) computationally practical: reused ideas get cheap, instant re-scoring instead of a fresh LLM call per candidate.

Worked Example

From a real K-BIG-2 run: a wizard profile of **K8,000 available capital, Lusaka (Kabulonga), skills in agriculture and selling & marketing, owning a smartphone and land** produced “**Kabulonga Green Basket**” (agriculture) -- A home-grown organic vegetable and herb subscription box service, grown on the owner's own plot and sold direct to affluent Kabulonga households via WhatsApp/Facebook ordering with doorstep delivery.

Reported numbers: N (target buyers) = 800, Q (purchases/year) = 48, P (avg. unit price) = K180 → raw demand = K6,912,000/year. Monthly revenue estimate K15,600, COGS K4,700 → gross margin 69.9%. Minimum capital required K3,000 against K8,000 available.

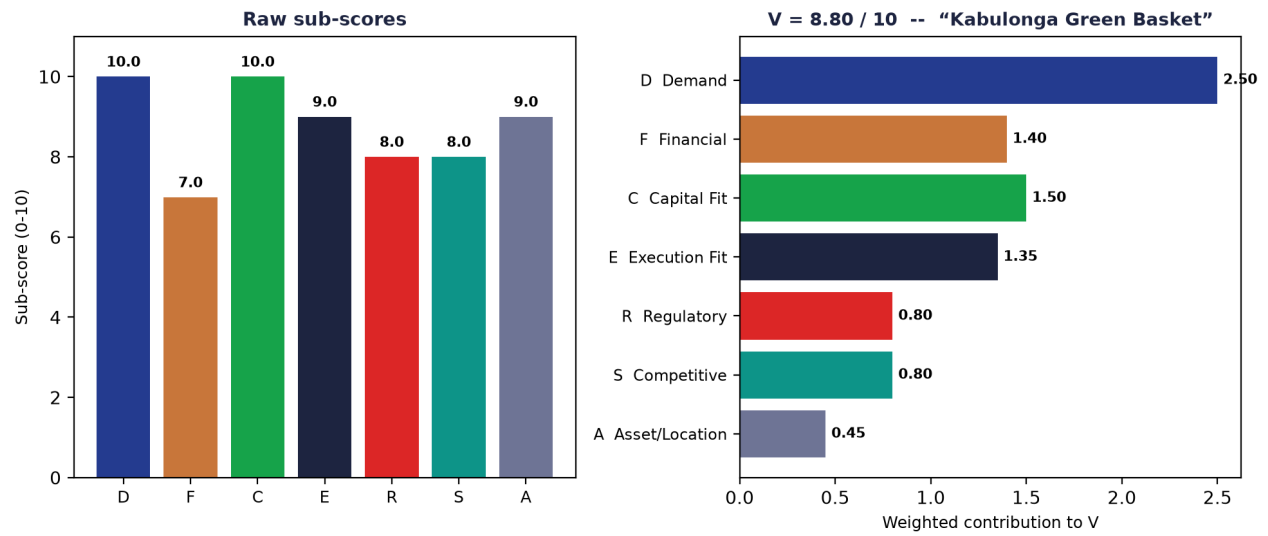


Figure 2 -- sub-scores and their weighted contribution to the final viability score ($V = 8.80 / 10$) for this idea, generated live against the real Claude Sonnet 5 structured-output pipeline during this build.

7. Methodology Assumptions & Design Decisions

In good conscience toward anyone evaluating this system, every place where the sprint specification left an implementation detail unstated is documented here explicitly, rather than silently decided.

- **Demand normalization.** The formula $D = N \cdot Q \cdot P \cdot (1+g)^t$ produces a raw Kwacha-scale number, but it is weighted alongside six 0-10 sub-scores. K-BIG-2 log-scales it against a tunable “reference_max” constant (an annual-demand figure considered excellent for a Zambian micro/small business, currently K5,000,000) in `category_cagr.json`: `score = min(10, log10(D+1) / log10(reference_max) * 10)`. This is the primary lever for recalibrating demand scoring as more real outcome data accumulates.
- **Cross-user re-ranking uses heuristics, not N extra LLM calls.** Re-scoring every historical idea in the dataset against a new requester via a fresh Claude call per idea would be slow and expensive at scale. Capital fit is recomputed with plain arithmetic; execution fit and asset/location advantage are recomputed via a fast skill/asset/location overlap ratio in Python. Only the 3 brand-new ideas in a given request get full LLM-assessed sub-scores.
- **Schema field consolidation.** The sprint's full field list (~70 fields) was found to compile to a structured-output validation grammar too large for the API once wrapped in a 3-idea array (a hard API limit, confirmed empirically during this build, not a design preference). K-BIG-2's schema keeps every field the viability formula consumes plus one representative field per remaining sprint-doc category, and folds the rest of that category's nuance (equipment, logistics, supplier dependency, seasonality, brand/cross-sell/upsell potential, health & safety notes) into a single free-text narrative field rather than one strict field each.
- **Model choice.** K-BIG-2 uses Claude Sonnet 5 with forced structured JSON output (guaranteeing a machine-parseable response, unlike K-BIG-1's regex-scraped markdown), matching K-BIG-1's existing Sonnet-tier model choice for a cost profile appropriate to a Zambian SME-focused product.
- **Dataset export is admin-only.** The full structured dataset -- K-BIG-2's core investor-facing asset -- is exportable as CSV via `GET /api/ideas/export`, gated behind a new platform-admin flag (no per-user personal information is included in the export).

8. Technology Stack

Layer	Technology
AI generation	Anthropic Claude Sonnet 5, forced JSON-schema structured output
Backend	FastAPI (Python), SQLAlchemy ORM, SQLite (dev) / PostgreSQL via Neon (prod)
Frontend	React 18 + Vite, custom 4-slide wizard component
Knowledge grounding	ChromaDB vector search (RAG) + hand-maintained Zambia economic constants
Geolocation	Browser GPS + haversine matching against KIP's own town coordinate registry

9. What This Means for Investors

K-BIG-2 turns every business-idea request into two assets: a scored, explainable recommendation for the entrepreneur, and a permanent addition to a growing, structured, proprietary dataset of Zambian micro/small-business viability data -- market sizing, financial assumptions, and risk profiles across dozens of real local business categories. That dataset compounds with every user, is fully exportable for due diligence, and is the foundation for future work: better demand calibration as real outcomes are logged, richer structured fields as the API's schema limits are worked around with multi-call generation, and eventually a fully data-driven (rather than LLM-estimated) viability model once enough real business outcomes have been observed through the platform.